E4 Educator v2.0

T13

LittleFS

Tier 2 · Tutorial 6 of 9

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- What LittleFS is and how it differs from NVS and SD card storage
- How to configure the partition table to allocate space for LittleFS
- How to upload files to LittleFS using PlatformIO's filesystem uploader
- How to read, write, and delete files on LittleFS
- How to serve a web dashboard from LittleFS over WiFi
- How to load smooth VLW fonts from LittleFS for the TFT display
- How to store and retrieve sensor history logs internally
- The three-storage decision framework: NVS, LittleFS, or SD

You will need:

- E4 Educator v2.0 board with ESP32 fitted
- T08 and T12 completed — TFT pipeline and WiFi working
- Button.h, Settings.h from previous tutorials

1 What is LittleFS?

LittleFS is a lightweight filesystem designed for microcontrollers that stores files in a dedicated partition of the ESP32's internal flash memory. It gives you a proper file system — with directories, file names, and arbitrary file content — without needing an external SD card.

It sits between NVS and SD card in capability. NVS holds small key-value settings. LittleFS holds medium-sized files like web pages, fonts, and config files. SD card holds large or growing datasets like sensor logs.

1.1 Three-storage decision framework

Property	NVS	LittleFS	SD card
Location	Internal flash	Internal flash	External microSD
Typical size	~20KB	~1–2MB	2GB–32GB
Always available	Yes	Yes	Card must be inserted
File system	No (key-value)	Yes (files + dirs)	Yes (FAT32)
Best for	Settings, calibration, counters	Web pages, fonts, config, small logs	Large logs, images, audio
Upload method	Preferences.putXxx()	PlatformIO uploader or at runtime	PC card reader

1.2 LittleFS versus SPIFFS

Older ESP32 projects use SPIFFS (SPI Flash File System). LittleFS replaces it entirely. LittleFS is more reliable (power-loss safe), faster, and supports directories. Always use LittleFS in new projects — never SPIFFS.

SPIFFS is deprecated

If you encounter older tutorials or code using SPIFFS.begin() or #include <SPIFFS.h>, replace it with LittleFS.begin() and #include <LittleFS.h>. The file operation API is identical — only the include and begin() call change. Migrating SPIFFS projects to LittleFS takes less than five minutes.

2 Partition table configuration

The ESP32's 4MB flash is divided into partitions. By default, the arduino partition table allocates most flash to the application and very little (or none) to a filesystem. You must use a partition table that includes a LittleFS partition.

2.1 Setting the partition table in platformio.ini

```
[env:e4_educator_v2]
platform   = espressif32@6.6.0
board      = esp32dev
framework  = arduino
monitor_speed = 115200

; Use the default partition table with LittleFS
; This gives ~1.4MB app + ~1.4MB filesystem
board_build.partitions = default_8MB.csv

; Or for standard 4MB boards, use this built-in table:
; board_build.partitions = min_spiffs.csv
; This gives ~1.9MB app + ~300KB filesystem (small but sufficient)

; For maximum filesystem space on 4MB boards:
; board_build.partitions = huge_app.csv  ; No OTA, max filesystem

; Upload filesystem image to LittleFS partition
board_build.filesystem = littlefs

lib_deps =
  bodmer/TFT_eSPI @ 2.5.43
  knolleary/PubSubClient @ 2.8
  adafruit/Adafruit AHTX0
  adafruit/Adafruit BusIO

; ... rest of build_flags as usual
```

Key point

The board_build.filesystem = littlefs line tells PlatformIO to use LittleFS format when uploading filesystem images. The board_build.partitions line selects the partition table. For E4 projects that also need OTA (T14), use min_spiffs.csv which reserves OTA space. For maximum filesystem size without OTA, use huge_app.csv.

2.2 Recommended partition choices for E4 projects

Partition table	App space	FS space	Use when
min_spiffs.csv	1.9MB	~300KB	OTA required (T14). Small filesystem sufficient.
default.csv	1.4MB	~1.4MB	Balanced. OTA + reasonable filesystem.
huge_app.csv	3MB	~1MB	Large app, no OTA. Maximum app space.

`default_8MB.csv`	~3MB	~3MB	8MB flash boards only — not standard ESP32 DevKit V1.

The E4 course uses min_spiffs.csv as the standard because T14 introduces OTA and the two tutorials share the same partition layout. The ~300KB filesystem is sufficient for web pages, fonts, and config files.

3 Uploading files with PlatformIO

PlatformIO can upload a folder of files to the LittleFS partition in one operation. Files placed in the data/ folder at the project root are bundled into a filesystem image and flashed to the LittleFS partition.

3.1 Project structure

```
E4_T13_LittleFS/
├── platformio.ini
├── src/
│   └── main.cpp
├── include/
│   └── Button.h
└── data/                    ← files here are uploaded to LittleFS
    ├── index.html
    ├── style.css
    ├── config.json
    └── fonts/
        └── Roboto24.vlw
```

3.2 Uploading the filesystem

1. Place your files in the data/ folder at the project root
2. In VS Code with PlatformIO, open the PlatformIO sidebar (ant icon)
3. Expand your project environment
4. Click "Upload Filesystem Image"
5. PlatformIO builds the LittleFS image from data/ and flashes it

Upload order matters

Upload the firmware first (normal Upload), then the filesystem (Upload Filesystem Image). The filesystem upload erases the LittleFS partition and reflashes it. If you upload the filesystem first and then the firmware, the OTA partition table may be disrupted on some configurations. Firmware first, filesystem second.

3.3 LittleFS initialisation

```
#include <Arduino.h>
#include <LittleFS.h>

void setup() {
  Serial.begin(115200);

  // Mount LittleFS — formatOnFail=true reformats if corrupted
  if (!LittleFS.begin(true)) {
    Serial.println("ERROR: LittleFS mount failed.");
    return;
  }

  // Report filesystem usage
  size_t total = LittleFS.totalBytes();
```

```
  size_t used  = LittleFS.usedBytes();
  Serial.printf("LittleFS: %u KB used of %u KB total\n",
                used / 1024, total / 1024);

  // List all files
  File root = LittleFS.open("/");
  File entry = root.openNextFile();
  while (entry) {
    Serial.printf("  %s  (%u bytes)\n",
                  entry.name(), entry.size());
    entry = root.openNextFile();
  }
}

void loop() {}
```

4 File operations on LittleFS

The LittleFS file API is identical to the SD file API from T10. The same read, write, append, and delete patterns apply — only the object name changes from SD to LittleFS.

```
#include <LittleFS.h>

// Read entire file into a String
String readFile(const char* path) {
  File f = LittleFS.open(path, "r");
  if (!f) { Serial.printf("Not found: %s\n", path); return ""; }
  String content = f.readString();
  f.close();
  return content;
}

// Write String to file (overwrites)
bool writeFile(const char* path, const String& content) {
  File f = LittleFS.open(path, "w");
  if (!f) { Serial.printf("Cannot write: %s\n", path); return false; }
  f.print(content);
  f.close();
  return true;
}

// Append line to file
bool appendLine(const char* path, const char* line) {
  File f = LittleFS.open(path, "a");
  if (!f) return false;
  f.println(line);
  f.close();
  return true;
}

// Check if file exists
bool fileExists(const char* path) {
  return LittleFS.exists(path);
}

// Delete file
void deleteFile(const char* path) {
  LittleFS.remove(path);
}

// Create directory
void makeDir(const char* path) {
  LittleFS.mkdir(path);
}
```

5 Serving a web dashboard from LittleFS

One of LittleFS's most powerful uses is hosting a web dashboard. With WiFi active, the E4 can serve HTML, CSS, and JavaScript from LittleFS to any browser on the same network. This gives the device a configuration and monitoring interface without needing Node-RED or any external server.

5.1 Web server library

Add the async web server libraries to platformio.ini:

```
lib_deps =
  bodmer/TFT_eSPI @ 2.5.43
  knolleary/PubSubClient @ 2.8
  adafruit/Adafruit AHTX0
  adafruit/Adafruit BusIO
  https://github.com/me-no-dev/ESPAsyncWebServer
  https://github.com/me-no-dev/AsyncTCP
```

5.2 data/index.html

Create this file in your project's data/ folder. It is a self-contained sensor dashboard that auto-refreshes every 5 seconds:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>E4 Dashboard</title>
  <style>
    body { font-family: Arial, sans-serif; background: #0a1628;
           color: #fff; margin: 0; padding: 20px; }
    h1   { color: #2E86AB; margin-bottom: 20px; }
    .card { background: #162040; border-radius: 12px;
            padding: 20px; margin: 10px 0; }
    .label { color: #8898aa; font-size: 0.9em; }
    .value { font-size: 2.5em; font-weight: bold; margin: 8px 0; }
    .temp  { color: #07E0A0; }
    .humid { color: #2E86AB; }
    .light { color: #FD9020; }
    .status { font-size: 0.8em; color: #8898aa; margin-top: 20px; }
  </style>
</head>
<body>
  <h1>E4 Educator Dashboard</h1>
  <div class="card">
    <div class="label">TEMPERATURE</div>
    <div class="value temp" id="temp">--</div>
  </div>
  <div class="card">
    <div class="label">HUMIDITY</div>
    <div class="value humid" id="humid">--</div>
```

```
  </div>
  <div class="card">
    <div class="label">LIGHT LEVEL</div>
    <div class="value light" id="light">--</div>
  </div>
  <div class="status" id="status">Connecting...</div>
  <script>
    async function refresh() {
      try {
        const r = await fetch("/api/sensors");
        const d = await r.json();
        document.getElementById("temp").textContent
          = d.temp.toFixed(1) + " °C";
        document.getElementById("humid").textContent
          = d.humidity.toFixed(1) + " %";
        document.getElementById("light").textContent
          = d.light + " %";
        document.getElementById("status").textContent
          = "Updated: " + new Date().toLocaleTimeString();
      } catch(e) {
        document.getElementById("status").textContent = "Error: " + e;
      }
    }
    refresh();
    setInterval(refresh, 5000);
  </script>
</body>
</html>
```

5.3 Web server in main.cpp

```
#include <Arduino.h>
#include <WiFi.h>
#include <LittleFS.h>
#include <ESPAsyncWebServer.h>
#include <Wire.h>
#include <Adafruit_AHTX0.h>

AsyncWebServer server(80);
Adafruit_AHTX0 aht;

float tempC = 0, humidity = 0;
int   lightPct = 0;

int readLDR() {
  long t = 0;
  for (int i = 0; i < 8; i++) { t += analogRead(LDR); delay(2); }
  return constrain(map(t/8, 200, 3800, 0, 100), 0, 100);
}

void readSensors() {
  sensors_event_t h, t;
  aht.getEvent(&h, &t);
```

```
  tempC    = t.temperature;
  humidity = h.relative_humidity;
  lightPct = readLDR();
}

void setupWebServer() {
  // Serve index.html from LittleFS at root URL
  server.on("/", HTTP_GET, [](AsyncWebServerRequest* req) {
    req->send(LittleFS, "/index.html", "text/html");
  });

  // JSON API endpoint for sensor data
  server.on("/api/sensors", HTTP_GET, [](AsyncWebServerRequest* req) {
    char json[80];
    snprintf(json, sizeof(json),
             "{\"temp\":%.1f,\"humidity\":%.1f,\"light\":%d}",
             tempC, humidity, lightPct);
    req->send(200, "application/json", json);
  });

  // Serve any file from LittleFS (CSS, JS, fonts etc.)
  server.serveStatic("/", LittleFS, "/");

  // 404 handler
  server.onNotFound([](AsyncWebServerRequest* req) {
    req->send(404, "text/plain", "Not found");
  });

  server.begin();
  Serial.printf("Web server running at http://%s\n",
                WiFi.localIP().toString().c_str());
}

void setup() {
  Serial.begin(115200);
  Wire.begin(I2C_SDA, I2C_SCL);

  if (!LittleFS.begin(true)) {
    Serial.println("LittleFS mount failed."); return;
  }
  Serial.printf("LittleFS: %u KB used\n",
                LittleFS.usedBytes() / 1024);

  if (!aht.begin(&Wire)) {
    Serial.println("AHT20 not found.");
  }

  WiFi.mode(WIFI_STA);
  WiFi.begin("lab_wifi", "fundrum1");
  Serial.print("Connecting WiFi");
  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 30) {
    delay(500); Serial.print("."); attempts++;
  }
```

```
  Serial.println();

  if (WiFi.status() == WL_CONNECTED) {
    setupWebServer();
    readSensors();
  } else {
    Serial.println("WiFi failed.");
  }
}

unsigned long lastRead = 0;

void loop() {
  // Update sensor readings every 5 seconds
  if (millis() - lastRead >= 5000) {
    lastRead = millis();
    readSensors();
    Serial.printf("T:%.1f H:%.1f L:%d\n",
                  tempC, humidity, lightPct);
  }
}
```

After uploading firmware and filesystem, open a browser on any device on the same network and navigate to the E4's IP address. The dark-themed sensor dashboard appears and updates every 5 seconds automatically.

Key point

The web server runs asynchronously in the background — it does not block loop(). Sensor readings update in the background and the API endpoint always returns the latest values. Any browser on the local network can access the dashboard simultaneously without interfering with the firmware.

6 Smooth fonts from LittleFS

TFT_eSPI supports smooth anti-aliased VLW fonts loaded from LittleFS at runtime. These look dramatically better than the built-in bitmap fonts — clean edges at any size, much more legible on a colour TFT display.

6.1 Generating VLW font files

VLW font files are generated using the Processing IDE font converter tool provided by TFT_eSPI. The process:

6. Download Processing IDE from processing.org
7. Open the font converter sketch: TFT_eSPI/Tools/Create_Smooth_Font/Create_font/
8. Set the font name, size, and character range in the sketch
9. Run the sketch — it generates a .vlw file in the sketch folder
10. Copy the .vlw file to your project's data/ folder
11. Upload filesystem to include the font

For the E4 course, two font files are recommended as the standard set:

File name	Size	Best use
`Roboto24.vlw`	24px	Body text, labels, secondary information
`Roboto48.vlw`	48px	Large values, temperature, key readings

6.2 Loading and using smooth fonts

```
#include <TFT_eSPI.h>
#include <LittleFS.h>

TFT_eSPI    tft;
TFT_eSprite spr = TFT_eSprite(&tft);

void setup() {
  // ... TFT and LittleFS init as normal ...

  // Load smooth font from LittleFS
  // Font stays loaded until loadFont("") is called
  tft.loadFont("Roboto48", LittleFS);

  // Draw with smooth font — must use drawString not print
  tft.setTextColor(TFT_WHITE, TFT_BLACK);
  tft.drawString("23.4 C", 10, 80);

  // Unload font when done to free RAM
  tft.unloadFont();

  // Load smaller font for labels
  tft.loadFont("Roboto24", LittleFS);
  tft.setTextColor(0xC618, TFT_BLACK);
  tft.drawString("Temperature", 10, 140);
  tft.unloadFont();
```

```
}

// Using smooth fonts in sprites (recommended for flicker-free)
void updateValueZone(float tempC) {
  spr.fillSprite(TFT_BLACK);

  // Load font into the sprite
  spr.loadFont("Roboto48", LittleFS);
  spr.setTextColor(0x07E0, TFT_BLACK);

  char buf[16];
  snprintf(buf, sizeof(buf), "%.1f C", tempC);
  spr.drawString(buf, 8, 8);
  spr.unloadFont();

  spr.loadFont("Roboto24", LittleFS);
  spr.setTextColor(0xC618, TFT_BLACK);
  spr.drawString("Temperature", 8, 64);
  spr.unloadFont();

  spr.pushSprite(0, 50);
}
```

drawString not print with smooth fonts

When using loadFont() smooth fonts, always use drawString(text, x, y) instead of setCursor() + print(). The smooth font engine uses a different rendering path from the bitmap font engine and setCursor() + print() does not work correctly with loaded VLW fonts.

7 JSON config file from LittleFS

Storing configuration as JSON in LittleFS is more structured than the key=value format used in T10. JSON config is easy to edit on a PC, supports nested data, and can be served and updated via the web dashboard.

7.1 data/config.json

```
{
  "device": {
    "name": "E4-Logger-01",
    "location": "Lab bench"
  },
  "display": {
    "brightness": 200,
    "timeout": 60
  },
  "alerts": {
    "tempHigh": 28.0,
    "humidHigh": 70.0
  },
  "network": {
    "broker": "192.168.1.79",
    "interval": 5000
  }
}
```

7.2 Parsing JSON with ArduinoJson

Add ArduinoJson to platformio.ini:

```
lib_deps =
  ; ... existing ...
  bblanchon/ArduinoJson @ 6.21.3
```

```
#include <ArduinoJson.h>
#include <LittleFS.h>

struct AppConfig {
  String devName     = "E4-Device";
  int    brightness  = 200;
  float  tempHigh    = 28.0;
  float  humidHigh   = 70.0;
  String mqttBroker  = "192.168.1.79";
  unsigned long interval = 5000;
};

AppConfig cfg;

bool loadConfig() {
  File f = LittleFS.open("/config.json", "r");
  if (!f) {
```

```
    Serial.println("config.json not found — using defaults.");
    return false;
  }

  StaticJsonDocument<512> doc;
  DeserializationError err = deserializeJson(doc, f);
  f.close();

  if (err) {
    Serial.printf("JSON parse error: %s\n", err.c_str());
    return false;
  }

  cfg.devName    = doc["device"]["name"]    | cfg.devName;
  cfg.brightness = doc["display"]["brightness"] | cfg.brightness;
  cfg.tempHigh   = doc["alerts"]["tempHigh"]  | cfg.tempHigh;
  cfg.humidHigh  = doc["alerts"]["humidHigh"] | cfg.humidHigh;
  cfg.mqttBroker = doc["network"]["broker"]   | cfg.mqttBroker;
  cfg.interval   = doc["network"]["interval"] | cfg.interval;

  Serial.printf("Config loaded: %s  T>%.1f  H>%.1f\n",
                cfg.devName.c_str(), cfg.tempHigh, cfg.humidHigh);
  return true;
}

bool saveConfig() {
  StaticJsonDocument<512> doc;
  doc["device"]["name"]           = cfg.devName;
  doc["display"]["brightness"]    = cfg.brightness;
  doc["alerts"]["tempHigh"]       = cfg.tempHigh;
  doc["alerts"]["humidHigh"]      = cfg.humidHigh;
  doc["network"]["broker"]        = cfg.mqttBroker;
  doc["network"]["interval"]      = cfg.interval;

  File f = LittleFS.open("/config.json", "w");
  if (!f) return false;
  serializeJsonPretty(doc, f);
  f.close();
  Serial.println("Config saved.");
  return true;
}
```

Key point

The | operator in ArduinoJson acts as a default value: doc["key"] | defaultValue returns the default if the key is absent or null. This means the config loader is safe for partial JSON files and first-boot situations where no config.json exists yet.

8 T13 summary

Concept	Key takeaway

LittleFS vs NVS vs SD	NVS: small key-value settings. LittleFS: medium files, always available. SD: large/growing data.
SPIFFS	Deprecated. Replace SPIFFS.begin() with LittleFS.begin() and #include <SPIFFS.h> with #include <LittleFS.h>. API is identical.
Partition table	board_build.partitions = min_spiffs.csv for OTA + LittleFS. board_build.filesystem = littlefs always.
Upload order	Firmware first (Upload), filesystem second (Upload Filesystem Image). Files in data/ folder are bundled automatically.
Web server	ESPAsyncWebServer serves files from LittleFS. /api/sensors returns JSON. serveStatic() handles all other files automatically.
Smooth fonts	loadFont("name", LittleFS) loads VLW font. Use drawString(text, x, y) not print(). unloadFont() frees RAM. Works in sprites.
ArduinoJson	StaticJsonDocument<512>, deserializeJson() to load, serializeJsonPretty() to save. The | operator provides safe defaults.
LittleFS.begin(true)	The true parameter formats the filesystem if it is corrupted or empty. Safe to use in setup() always.

What's next

- T14 — OTA updates: with WiFi, LittleFS, and the web server established, adding over-the-air firmware updates via ElegantOTA is the final piece of the networked device puzzle. A device that can update itself over WiFi without a USB cable is production-ready.